สรุปสอบ: Jest · API Testing · MVC · Cookies

สารบัญ — ใช้ศัพท์เทคนิคภาษาอังกฤษตามสไลด์

1	**Unit Testing with Jest** Bug และ Debugging · Error Types · รู้ได้อย่างไรว่า function ทำงานถูก? · Software Testing · Semantic Versioning (semver.org) — X.Y.Z · Unit Testing · Manual vs Automation Testing · Test Driven Development (TDD) · การออกแบบ Test Case · Jest: Setup (jestjs.io) · AAA Pattern · ตัวอย่าง validateUsername · describe() และ it() · Matchers · Quiz ในสไลด์ · Project Work	**หน้า 2**
2	**API Testing** API Testing อยู่ตรงไหน · Function และ Spaghetti code · Calling an API — Accept header · Exercise: ส่ง HTTP GET ด้วย curl · Postman · API Types · HTTP methods (CRUD) และ Status codes · Lab: todo-list-testing · Testing Secure APIs · Testing GET calls · Testing PUT calls — สิ่งที่พบ · Testing DELETE / authorization · Assignment (2 points) · Checklist ทดสอบ API	**หน้า 6**
3	**Model-View-Controller (MVC)** MVC Pattern คืออะไร · หน้าที่ของแต่ละส่วน · ตัวอย่าง: Login เข้าสู่ระบบ · Lab: เปลี่ยน Project เดิมให้เป็นระเบียบ · ขั้นที่ 1: แยก Config ฐานข้อมูล · ขั้นที่ 2: สร้าง Route ของ product · ขั้นที่ 3: แยก Controller · ขั้นที่ 4: แยก Model · View ใน Express.js · Embedded JavaScript Templates (EJS) · Lab ต่อ: แสดงสินค้าด้วย EJS · Static files (CSS)	**หน้า 8**
4	**Cookies, Sessions และ Password Hashing** Cookie คืออะไร? · ประโยชน์ของ Cookie · ข้อมูลที่ไม่ควรเก็บใน Cookies · Demo: Session hijacking (the-internet.herokuapp.com) · Cookie attributes · Project I — Session-cookie login · Project II — เก็บรหัสผ่านด้วย bcrypt · Plaintext vs Encrypted vs Hash · bcrypt.hash(password, 10)	**หน้า 12**
5	**เครื่องมือที่ใช้ (Tools)** ภาพรวม: เครื่องมือ → ใช้ในหัวข้อไหน · Node.js / npm · Jest · curl · Postman · Chrome DevTools (F12) · API documentation (/docs) · Express ecosystem · Docker Compose · VS Code extensions · nodemon / node --watch · เว็บที่ใช้ทดลอง	**หน้า 17**
6	**นอกสไลด์ (Beyond the slides)** 1. req.params vs req.query vs req.body · 2. XSS และ CSRF · 3. ทดสอบโค้ด async ใน Jest · 4. PUT vs PATCH	**หน้า 20**

1 · Unit Testing with Jest

Bug และ Debugging

- **First law of software quality** (มุกในสไลด์): `Errors = (more code)²` → โค้ดยิ่งเยอะ error ยิ่งเพิ่มแบบทวีคูณ
- **ประวัติ Bug**: มีแมลง (bug) เข้าไปติดใน **Relay** ของเครื่อง **Computer Mark II** ทำให้เครื่องขัดข้อง ทีมงานจึงเรียกปัญหาต่าง ๆ ว่า "Bug" และใช้คำนี้ในวงการคอมพิวเตอร์ต่อมา
- **Bug** = จุดบกพร่อง/ข้อผิดพลาดในโปรแกรมที่เกิดจากโปรแกรมเมอร์ ทำให้คอมพิวเตอร์ทำงานไม่ถูกต้อง
- **Debug** = การแก้ไขจุดบกพร่องให้โปรแกรมทำงานปกติ

Error Types

- **Syntax error** = พิมพ์ชุดคำสั่งผิดรูปแบบภาษา
  ตัวอย่างในสไลด์:

```
const greeting = "Hello World;      // ปิด quote ไม่ครบ
if age === 20 { ... }               // ไม่มีวงเล็บ ( )
const user = { name: "John"
  age: 30 };                        // ขาด comma
const class = "Programming";        // class เป็น reserved word
const add = x, y => x + y;          // ต้องเป็น (x, y) =>
```

- **Run-time error** = เกิดขึ้น "ระหว่างที่โปรแกรมกำลังทำงาน (Execution)"

```
function displayCheckoutPrice(price) {
  const formattedPrice = price.toFixed(2);
  console.log("Total to pay: $" + formattedPrice);
}
displayCheckoutPrice("Free"); // string ไม่มี toFixed -> TypeError
```

- **Logic error** = โปรแกรมรันได้ แต่ผลลัพธ์ผิดไปจากที่ออกแบบไว้

```
const average = mathScore + scienceScore / 2;
// ผิด precedence: ต้องเป็น (mathScore + scienceScore) / 2
getAverageScore(80, 90); // ได้ 125 แทน 85
```

รู้ได้อย่างไรว่า function ทำงานถูก?

```
function canEnterConcert(age, ticket) {
  return age >= 18 && ticket;
}
console.log(canEnterConcert(14, true));  // false
console.log(canEnterConcert(25, false)); // false
console.log(canEnterConcert(18, true));  // true
```

การดูผลด้วย `console.log` เองคือ manual testing → จึงต้องมี automated test

Software Testing

- **Software testing** = กระบวนการประเมินการทำงานของซอฟต์แวร์ ว่าตรงตาม requirements หรือไม่ และหา defects เพื่อให้ได้ผลิตภัณฑ์คุณภาพ
- **ใครทำ Testing**: Software Tester, Software Developer, Project Lead/Manager, End User
- **Testing Types**:
  - **Unit Testing** → **Integration Testing** → **System Testing**
  - **User Acceptance Testing (UAT)** = ยืนยันว่าซอฟต์แวร์ใช้งานได้จริงสำหรับผู้ใช้
  - **Regression Testing** = ตรวจหา side effects ที่ไม่คาดคิดหลังแก้โค้ด
- โครงสร้าง: Unit Test หลายตัว → รวมเป็น Integration Test → รวมเป็น System Test
- **Test Pyramid** (ล่าง→บน): **Unit** → **Component** → **Integration** → **API** → **UI**
  ล่าง = Isolated + Faster ; บน = Integrated + Slower
- **Non-Functional Requirement Testing**: Performance, Volume, Security, Reliability, Availability, Reusability, Flexibility ฯลฯ

Semantic Versioning (semver.org) — X.Y.Z

ส่วน	ความหมาย
Major (X)	เปลี่ยน/เพิ่ม API หลักจำนวนมาก **ไม่รองรับ** API เวอร์ชันเก่า → โค้ดเก่ากับใหม่ทำงานร่วมกันไม่ได้
Minor (Y)	เพิ่มฟังก์ชันใหม่เล็กน้อย ไม่กระทบโค้ดปัจจุบัน (**Backward compatibility**)
Patch (Z)	ปรับปรุง/แก้ไข Bugs ของเวอร์ชันหลักปัจจุบัน

Unit Testing

- แบ่งระบบออกเป็น **units** ตามตรรกะ ขนาดตั้งแต่ function เล็ก ๆ จนถึงทั้ง class
- ปกติทำโดย **developers** ระหว่างการพัฒนา ผลออกมาเป็น **pass/fail**
- ตัวอย่าง: `validateScore()`, `validatePassword()`, `calculateDiscount()`, `calculateShipping()`, `reverseString()`, `isPalindrome()`, `calculateBMI()`, `validateUsername()`

Manual vs Automation Testing

ปัญหา Manual Testing: น่าเบื่อ/ซ้ำซาก, human error, เสียเวลา (ระบบจริงอาจมี 2,000 – 3,000 test cases)

Automation testing = ใช้ tools, scripts, software รัน test cases โดยทำ actions ที่กำหนดไว้ซ้ำ ๆ

	Manual	Automated
Reliability	ไม่แม่นยำเสมอ เพราะ human error	น่าเชื่อถือกว่า ทำโดย tools/scripts
Time	ใช้เวลามาก	เร็วกว่ามาก
ใช้เมื่อ	Exploratory, Usability, Ad-hoc Testing	Test cases ที่ต้องทำซ้ำบ่อย (functional/non-functional)

Test Driven Development (TDD)

"เราจะไม่เขียน Code จนกว่าจะเขียน Test" → วงจร **Red (fail) → Green (pass) → Refactor**

Unit testing frameworks: **Jest**, Vitest, Node.js Native Test Runner, Mocha ฯลฯ

การออกแบบ Test Case

คำถามที่ต้องถาม:

- อะไรที่ควรทำงานได้ปกติ? อะไรที่ควรถูกปฏิเสธ?
- มี **boundaries** (min/max/threshold) ไหม? มี **branches** (if/else) ไหม?
- มีอะไรที่อาจ fail หรือ **throw error/exception** ไหม?

Requirement: Password must contain at least 8 characters.

```
// Code I  (ผิด!)                    // Code II (ถูก)
return password.length > 8;     return password.length >= 8;
```

ทดสอบแค่ 10 ตัวกับ 3 ตัว → **ทั้งสองโค้ดผ่าน** หาบั๊กไม่เจอ ต้องทดสอบที่ขอบเขต (8 ตัว) จึงเห็นความต่าง

- **Positive Test Cases** = ทดสอบสถานการณ์ที่ input/operation **เป็นไปตาม**เงื่อนไขที่ระบบยอมรับ (10, 15, 20 chars → PASS)
- **Negative Test Cases** = ทดสอบสถานการณ์ที่ input/operation **ไม่เป็นไปตาม**เงื่อนไข (6, 4, 1 chars → ระบบปฏิเสธถูกต้อง = test PASS)
- **Boundary Value**: ทดสอบที่ขอบ เช่น 6, 7 = Fail | **8** = boundary | 9, 10 = Pass
  หมายเหตุ: Bug จำนวนมากเกิดบริเวณเงื่อนไขขอบเขต เช่น `<`, `<=`, `>`, `>=`

Length	6	7	8	9	10
Expected	false	false	true	true	true

Negative inputs	
`null`, `12345` (number), `""`, `"john doe"`	Expected: false

Jest: Setup (jestjs.io)

```
mkdir my-testing
cd my-testing
npm init -y
npm install -D jest
code .
// package.json
{ "scripts": { "test": "jest" } }
// รัน
npm test
```

```
// sum.js
function sum(a, b) { return a + b; }
module.exports = sum;

// sum.test.js
const sum = require("./sum");
test("2 + 3 = 5", () => {
  expect(sum(2, 3)).toBe(5);
});
```

- `test(...)` = ใช้สร้าง Test Case `"2 + 3 = 5"` = ชื่อ Test Case
- `expect(...)` = บอก Jest ว่า "เราคาดหวังผลลัพธ์นี้" `.toBe(5)` = ค่าที่คาดหวัง
- **Test Suite** = กลุ่มของ tests (ไฟล์ `.test.js`) → 3 ไฟล์ = "Test Suites: 3 passed, 3 total"
- **Test Case** = แต่ละกรณีที่เขียนด้วย `test()` หรือ `it()` → 3 อัน = "Tests: 3 passed, 3 total"

AAA Pattern

- **Arrange** (เตรียมข้อมูล) → **Act** (เรียกฟังก์ชันที่ต้องการทดสอบ) → **Assert** (ตรวจผล)

```
// calculateDiscount.js
function calculateDiscount(price, member) {
  if (member) { return price * 0.9; }
  return price;
}
module.exports = calculateDiscount;

// calculateDiscount.test.js
const calculateDiscount =
require("./calculateDiscount");
test("member should receive 10% discount", () =>
{
  // Arrange
  const price = 1000;
  const member = true;
  // Act
  const result = calculateDiscount(price,
member);
  // Assert
  expect(result).toBe(900);
});
```

ตัวอย่าง validateUsername

Requirement: Username must contain 5–20 characters, contain only letters, numbers, and underscore, must not contain spaces, and must start with a letter.

No	Input	Test Case	Exp.
P1	"john1"	Minimum length 5	true
P2	"john_doe"	Contains underscore	true
P3	"User_123"	Letters + Numbers + Underscore	true
P4	"abcde12345"	Alphanumeric	true
P5	"abcdefghijklmnopqrst"	Maximum length 20	true
N1	"john"	Only 4 characters	false
N2	"abcdefghijklmnopqrstu"	21 characters	false
N3	"1john"	Starts with a number	false
N4	"john doe"	Contains a space	false
N5	"john@123"	Invalid character	false

```
function validateUsername(username) {
  if (typeof username !== "string") return
false;   // ต้องเป็น string
  if (username.length < 5 || username.length >
20) return false;
  if (!/^[A-Za-z]/.test(username)) return false;
// ขึ้นต้นด้วยตัวอักษร
  if (!/^[A-Za-z0-9_]+$/.test(username)) return
false; // letters/digits/_ ไม่มี space
  return true;
}
module.exports = validateUsername;
```

describe() และ it()

- `describe()` ใช้**จัดกลุ่ม** test cases ที่เกี่ยวข้องกัน และมีหลาย `describe()` ในไฟล์เดียวได้ (เช่น describe: validateUsername, describe: isAdult)
- ภายใน describe ใช้ได้ทั้ง `test()` หรือ `it()` (เหมือนกัน) — `it` อ่านเหมือนประโยค: "it should accept a valid username"

```
const validateUsername =
require("./validateUsername");
describe("validateUsername", () => {
  // Positive Test Cases
  it("should accept a valid username", () => {

expect(validateUsername("john1")).toBe(true);
  });
  // Negative Test Cases
  it("should reject a username shorter than 5
characters", () => {

expect(validateUsername("john")).toBe(false);
  });
});
```

Matchers

Matchers ใช้ตรวจว่า "ค่าที่ได้จริง" ตรงกับ "ค่าที่คาดหวัง" หรือไม่:

`expect(actualValue).matcher(expectedValue)`

อ่านว่า "I expect add(2,3) to be 5"

Matcher	ใช้เมื่อ / ตัวอย่าง
`toBe()`	ค่า primitive เท่ากันแบบเป๊ะ (ใช้ `Object.is`): number, string, boolean, `null`, `undefined` `expect(getGrade(85)).toBe("A")`, `expect(findUserById(999)).toBe(null)`, `expect(getMiddleName()).toBe(undefined)`
`toEqual()`	**Object / Array** — เปรียบเทียบทุก field แบบ recursive (รวม nested) `expect(getFailedStudents()).toEqual([])`, `expect(getSettings()).toEqual({})`, `expect(getStudent()).toEqual({name:"Alice", subjects: ["Math","Programming"]})`
`toBeTruthy()`	true, number ที่ไม่ใช่ 0 (รวมเลขติดลบ, ทศนิยม), string ไม่ว่าง (รวม `"false"`), object, array
`toBeFalsy()`	`false`, `0`, `""` (empty string), `null`, `undefined`, `NaN`
`toContain()`	มี item ใน array หรือมี substring ใน string `expect(getUserRoles()).toContain("admin")`, `expect(getErrorMessage()).toContain("Invalid")`
`toThrow()`	ต้องห่อด้วย function เสมอ `expect(() => divide(10, 0)).toThrow();` (ไม่สนข้อความ) `.toThrow("Cannot divide by zero")`, `.toThrow(TypeError)`, `.toThrow(RangeError)`, `.toThrow(new Error("Username is required"))`
`.not`, `toStrictEqual`, `toBeCloseTo`	กลับเงื่อนไข; เทียบแบบเข้มกว่า toEqual (สน key ที่เป็น undefined และ type); ทศนิยม (0.1+0.2)

Quiz ในสไลด์

#	คำถาม	คำตอบ
1	Unit Test คืออะไร?	Test small unit independently
2	ข้อใดคือ assertion ใน `expect(add(2,3)).toBe(5)`?	ทั้งคำสั่ง expect + matcher คือ assertion; `toBe(5)` คือ matcher (สไลด์ไม่ได้เฉลย)
3-4	Object ควรใช้ matcher ใด?	`toEqual()`
5	Negative test ของ "รหัสผ่าน ≥ 8 ตัว"	Password 3 characters
6	Negative test ของ ATM "ถอนสูงสุด 20,000"	ลองถอน 25,000 บาท
7	`add(a, b)` ต้องทดสอบ Boundary value ไหม?	สไลด์ไม่ได้เฉลย — ฟังก์ชันไม่มีเงื่อนไข/threshold จึงโดยทั่วไป **ไม่ต้อง** (ความเห็นของผู้สรุป ควรตรวจกับอาจารย์)

Project Work

นำเสนอตัวอย่างสรุปผลการทดสอบระบบ **ขั้นต่ำ 5 features** เช่น `validateUsername()` (valid username, invalid character, length boundary), `canAccessAdmin()` (Admin, normal user)

2 · API Testing

API Testing อยู่ตรงไหน

- Test Pyramid: Unit → Component → Integration → **API** → UI — API test อยู่ระหว่าง Integration กับ UI (integrated มากกว่า unit แต่เร็วกว่า UI)
- สถิติในสไลด์: Akamai (State of the Internet / Security Report) พบว่า **83%** ของ traffic ที่เห็นเป็น **API driven**
- การเรียก API = ส่ง **HTTP request** (method + URL + headers + body) แล้วตรวจ **response** (status code, headers, body)

Function และ Spaghetti code

- **Function** = ส่วนของโปรแกรมที่ทำหน้าที่บางอย่าง เรียกใช้ซ้ำได้ (Input → Function → Output, ต่อกันหลาย function ได้)
- **ข้อเสียของ Imperative programming**: function เรียกกันไปมาจนพันกัน → **Spaghetti code** = โค้ดที่ไม่มีโครงสร้าง ดูแลรักษายาก
- ตัวอย่าง API สาธารณะ: `https://api.mathjs.org/` (ส่ง expression ไปคำนวณ)

Calling an API — Accept header

Endpoint ใช้ URL เดียวกับ browser แต่รูปแบบ response เปลี่ยนตาม HTTP **Accept** header:

Accept	Response
`text/html`	HTML (default)
`application/json`	JSON
`text/plain`	Plain text

Note: request ผ่าน **curl** ที่ไม่ตั้ง Accept header จะได้ `text/plain` เป็น default (ของ icanhazdadjoke.com)

```
curl -H "Accept: application/json"
https://icanhazdadjoke.com/
curl -H "Accept: text/plain"
https://icanhazdadjoke.com/
```

Exercise: ส่ง HTTP GET ด้วย curl

```
curl https://icanhazdadjoke.com/
curl https://randomuser.me/api/
curl https://pokeapi.co/api/v2/pokemon/pikachu
curl https://httpbin.org/get
curl https://dog.ceo/api/breeds/image/random
```

Postman

- `postman.com` — มี desktop app แต่ Lab นี้**ไม่ต้องติดตั้ง** ใช้ web: `https://web.postman.co/`

API Types

Type	รายละเอียด / ตัวอย่างในสไลด์
REST (Representational State Transfer)	ใช้ HTTP methods + URL, ส่วนใหญ่ส่ง JSON
SOAP (Simple Object Access Protocol)	ใช้ XML; ตัวอย่าง `webservices.oorsprong.org/websamples.countryinfo/CountryInfoService.wso`
GraphQL	client ระบุ field ที่ต้องการเอง; `graphql.org`, ตัวอย่าง Apollo Studio "countries"

HTTP methods (CRUD) และ Status codes

Method	Action	Success
GET	อ่านข้อมูล (read)	200 OK
POST	สร้าง (create)	201 Created
PUT	แก้ไข (update)	200 OK
DELETE	ลบ (delete)	200 / 204

Code	ความหมาย
400	Bad Request — input ไม่ถูกต้อง
401	**Unauthorized** = ยังไม่ *authenticate* (ไม่มี/credential ผิด) — "คุณคือใคร?"
403	**Forbidden** = login แล้วแต่ *ไม่มีสิทธิ์* — "คุณทำอะไรได้บ้าง?"
404	Not Found
500	Internal Server Error — ส่วนใหญ่คือ **bug** ฝั่ง server (Assignment: หา 500 errors)

Lab: todo-list-testing

- Repo: `github.com/djwester/todo-list-testing` → รันด้วย `$ make run-dev` แล้วตั้ง port เป็น **Public**
- เปิด **Chrome DevTools** ดู request ที่หน้าเว็บส่ง (copy มาวางใน Postman แล้วแก้ description เพื่อเพิ่มข้อมูล)
- ทดลองดึงข้อมูลผ่าน task id ด้วย **GET**; แก้ status ผ่าน task id (ต้องพิมพ์ค่าให้ตรง เช่น "In Progress")
- ลบข้อมูลโดยไม่ login → ถูกปฏิเสธเพราะไม่มี **Authenticated**
- **API Discovery Challenge**: เปิด `<Your-URL>/docs` เพื่อดู API documentation → **Login** → **Copy Token** → วาง token ในช่อง Authorize
- **Assignment**: login ด้วย user1 / user2 / anonymous (password `12345`) → เพิ่ม Task ใหม่ → Edit ข้อความ → เปลี่ยน status **Draft** → **In Progress**
- บาง endpoint แสดง "There is no documentation for this endpoint." — ยังต้องทดสอบได้

Testing Secure APIs

- เข้าถึง secure endpoint อย่างไร? → **Authentication: Who are you?** **Authorization: What can you do?**
- มี Authorization หลายแบบ (Basic, Bearer token ฯลฯ)
- **Basic Auth**: กรอก username และ password → ส่งเป็น `Authorization: Basic <encoded-value>` โดย encoded-value = Base64 ของ `username:password` (ถอดได้ที่ base64decode.org)
- **Note: Base64 ไม่ใช่ encryption** มันเป็นเพียง **encoding** → Basic Auth ควรใช้ผ่าน **HTTPS** เพื่อป้องกัน credential ระหว่างส่งข้อมูล

Testing GET calls

No	Method	URL
1	GET	`<base-url>/tasks/1`
2	GET	`<base-url>/user`
3	GET	`<base-url>/user/user1`
4	GET	`<base-url>/user/me`
5	GET	`<base-url>/tasks?status=In%20Progress`
6	GET	`<base-url>/tasks?search=Keep`

ข้อ 5-6 ใช้ **Query string parameters** (`?key=value`, `%20` = space)

Testing PUT calls — สิ่งที่พบ

- **PUT** `<base-url>/tasks/1` = แก้ไขข้อมูล task
- ลองเปลี่ยน id ใน body จาก 1 เป็น 100 → **ID จะไม่ถูกแก้ไข** ระบบใช้ ID จาก URL
- PUT ข้อมูล user แล้วลบ email → **Email was deleted** (ถ้าไม่ใส่ข้อมูลใน body ข้อมูลจะถูกลบ) → **นี่คือ Bug**
- login เป็น user1 แล้วแก้ข้อมูล user อื่น → ถูกปฏิเสธ = **ถูกต้อง** (แก้ข้อมูลคนอื่นไม่ได้)

Testing DELETE / authorization

1. สร้าง task ใหม่ด้วย **user1** แล้วตรวจสอบว่าสร้างสำเร็จ (GET)
2. Login เป็น **user2** แล้วใช้ Token ของ user2
3. user2 ไป delete task ของ user1 → **ระบบไม่อนุญาต (ถูกต้อง)**
4. ลบ task id 1 ที่ **anonymous** เป็นผู้สร้าง

Assignment (2 points)

Break the API Challenge: พยายามหา **500 errors** ใน application (เช่น ส่ง body ผิด type, field ขาด, ค่ายาวมาก)

Checklist ทดสอบ API

- **Status code** ถูกต้อง? **Body** (field/ค่าใน JSON)? **Headers** (Content-Type)?
- **Positive** (input ถูก) และ **Negative** (field ขาด, type ผิด, body ว่าง)
- **Boundaries** (ความยาว min/max, id ที่ไม่มีอยู่)
- **Auth**: ไม่มี token → 401; ข้อมูลของคนอื่น/role ไม่พอ → 403; token ถูก → 200
- **Side effects**: หลัง POST/PUT/DELETE ให้ GET ซ้ำเพื่อยืนยันสถานะ

> **จำไว้**: 401 = ปัญหา *authentication* (ไม่ได้ login / credential ผิด) 403 = ปัญหา *authorization* (login แล้วแต่ไม่มีสิทธิ์) Base64 ≠ encryption ส่ง input ผิดแล้วได้ 500 = bug

3 · Model-View-Controller (MVC)

MVC Pattern คืออะไร

- **Software Architecture Pattern** สำหรับพัฒนาแอป ที่แบ่งหน้าที่ของโค้ดออกเป็น **3 ส่วนหลัก** → โค้ดเป็นระเบียบ แก้ไขง่าย ทำงานร่วมกันในทีมสะดวก
- **ทำไมต้อง MVC?** ตอนแรกที่ฟังก์ชันน้อยอาจไม่เห็นประโยชน์ แต่เมื่อฟังก์ชันเยอะขึ้น หรือกลับมาอ่านโค้ดที่ไม่ได้จัดรูปแบบ จะปวดหัวมาก → การเขียนเป็น pattern ช่วยให้อ่านง่าย แยกการทำงานชัดเจน
- **แนวคิด**: ใช้หลัก OOP แบ่งเป็น 3 objects คือ model, view, controller — กติกาคือ **แยกการทำงานชัดเจน ห้ามก้าวก่ายงานกันเด็ดขาด**

หน้าที่ของแต่ละส่วน

ส่วน	หน้าที่
Model	จัดการ**ข้อมูล**ทั้งหมด (Handle Data Logic, Interacts with Database) เตรียมข้อมูลที่เหมาะสมไว้และทำงาน**เมื่อ Controller ร้องขอเท่านั้น**
View	จัดการ**หน้าตา**ทั้งหมด / ส่วนติดต่อผู้ใช้ (**User Interface**) รับคำสั่งจาก Controller และเป็นตัวกลางให้ผู้ใช้ติดต่อกับ Controller (Handles Data Presentation, Dynamically Rendered)
Controller	**มันสมองและศูนย์กลาง** ทุกส่วนติดต่อผ่าน Controller และรอคำสั่งจากมัน จัดการ **logic** ทั้งหมด (Handles Request Flow, Never Handles Data Logic)

แผนภาพในสไลด์: View ส่ง input ให้ Controller (e.g. user clicks "add to cart") → Controller **Manipulates** Model (add item) → Model **Updates** View (show added item)

ตัวอย่าง: Login เข้าสู่ระบบ

1. ผู้ใช้ติดต่อ **V** → V ไปบอก **C**
2. C ส่งข้อมูลไปหา **M** เพื่อตรวจสอบความถูกต้อง
3. M ติดต่อ**ฐานข้อมูล**ตรวจสอบ แล้วส่งผลกลับไปหา C
4. C ดำเนินการต่อ 2 กรณี:
   - **สำเร็จ**: C สั่ง V เปลี่ยนเป็นหน้าเว็บหลัก
   - **ไม่สำเร็จ**: C สั่ง V บอกผู้ใช้ว่ารหัสผ่านผิดพลาด

สรุป: Model = จัดเก็บและจัดการกับข้อมูล; View = แสดงผล/ติดต่อกับผู้ใช้ (UI) รับคำสั่งจาก Controller; Controller = ตัวกลางควบคุมการรับ-ส่งข้อมูลระหว่าง Model และ View
Framework แบบ MVC: Flask, Django, Express, Rails, Laravel, CodeIgniter

Lab: เปลี่ยน Project เดิมให้เป็นระเบียบ

ดาวน์โหลด project จาก Google Drive ในสไลด์ → เดิมมีแค่ `.env`, `docker-compose.yaml`, `index.ejs`, `index.js`, `package.json`, `README` (route + SQL อยู่ใน index.js ทั้งหมด)

สร้าง Folder: `configs`, `controllers`, `models`, `public`, `routes`, `views`

```
project/
 configs/db.js            // เชื่อมต่อ DB
 routes/productRoutes.js  // URL -> controller
 routes/webRoutes.js      // หน้าเว็บ (EJS)
 controllers/productController.js
 models/productModel.js   // SQL queries
 views/products.ejs       // template
 public/css/              // static files
 .env  docker-compose.yaml  index.js
package.json
```

ขั้นที่ 1: แยก Config ฐานข้อมูล

```
// configs/db.js
const mysql = require('mysql2');
const dotenv = require('dotenv');
dotenv.config();
const db = mysql.createConnection({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});
db.connect(err => {
  if (err) throw err;
  console.log('Connected to MySQL');
});
module.exports = db;

// index.js (เชื่อม db)
const db = require('./configs/db');
```

ขั้นที่ 2: สร้าง Route ของ product

- สร้าง `routes/productRoutes.js` แล้ว**ย้าย route ทั้งหมดจาก index.js** มาใส่
- **อย่าลืมเปลี่ยน `app` เป็น `router` ให้หมด** (`app.get` → `router.get`)
- ใส่ `module.exports = router;` ต่อท้าย เพื่อดึงไฟล์ไปใช้งาน

```
// index.js
const express = require('express');
const app = express();
const PORT = 3000;
const productRoutes =
require('./routes/productRoutes');
app.use(express.json());
app.use('/api', productRoutes); // ทุก route ใน
productRoutes มี prefix /api
app.listen(PORT, () =>
  console.log(`Server is running at
http://localhost:${PORT}`));
```

ทดสอบ: `node .\index.js` → เปิด `http://localhost:3000/api/products` ได้ JSON

ปัญหาที่เหลือ: ในไฟล์ route **ยังมีส่วนของ Models กับ Controllers อยู่** (SQL + logic)

ขั้นที่ 3: แยก Controller

สร้าง `controllers/productController.js` → **ย้ายส่วนที่ไม่ใช่เส้นทาง** (ฟังก์ชัน `(req, res) => {...}`) ไปไว้ใน Controller

```
// routes/productRoutes.js (หลังแยก)
const express = require('express');
const router = express.Router();
const productController =
require('../controllers/productController');
router.get('/products',
productController.getAllProducts);
router.get('/products/:id',
productController.getProductById);
router.get('/products/search/:keyword',
productController.searchProducts);
router.post('/products',
productController.createProduct);
router.put('/products/:id',
productController.updateProduct);
router.delete('/products/:id',
productController.softDeleteProduct);
router.put('/products/restore/:id',
productController.restoreProduct);
module.exports = router;
```

ขั้นที่ 4: แยก Model

หมายเหตุ: สไลด์สร้าง folder ชื่อ `configs` แต่บางไฟล์เขียน `require('../config/db')` (ไม่มี s) → path ต้องตรงกับชื่อ folder จริง ในสรุปนี้ใช้ `../configs/db`

Controller ยังมี SQL อยู่ → ต้องแยก models ออกมา และติดตั้ง `npm install uuid` เพื่อสร้าง ID สินค้าใหม่

```
// models/productModel.js
const db = require('../configs/db');
const { v4: uuidv4 } = require('uuid');
const Product = {
  getAll: (callback) => {          // ดึงสินค้า
ทั้งหมด
    const query = 'SELECT * FROM products WHERE
is_deleted = 0';
    db.query(query, callback);
  },
  getById: (id, callback) => {     // ดึงตาม ID
    const query = 'SELECT * FROM products WHERE
id = ? AND is_deleted = 0';
    db.query(query, [id], callback);
  },
  searchByKeyword: (keyword, callback) => {
    const searchTerm = `%${keyword}%`;
    const query = 'SELECT * FROM products WHERE
name LIKE ? AND is_deleted = 0';
    db.query(query, [searchTerm], callback);
  },
  create: (productData, callback) => {
    const { name, price, discount, review_count,
image_url } = productData;
    const id = uuidv4();           // สร้าง ID ใหม่
    const query = 'INSERT INTO products (id,
name, price, discount, review_count, image_url)
VALUES (?, ?, ?, ?, ?, ?)';
    db.query(query, [id, name, price, discount,
review_count, image_url], callback);
  },
  update: (id, productData, callback) => {
    const { name, price, discount, review_count,
image_url } = productData;
    const query = 'UPDATE products SET name = ?,
price = ?, discount = ?, review_count = ?,
image_url = ? WHERE id = ?';
    db.query(query, [name, price, discount,
review_count, image_url, id], callback);
  },
  softDelete: (id, callback) =>
    db.query('UPDATE products SET is_deleted = 1
WHERE id = ?', [id], callback),
  restore: (id, callback) =>
    db.query('UPDATE products SET is_deleted = 0
WHERE id = ?', [id], callback),
};
module.exports = Product;
```

```
// controllers/productController.js (หลังแยก Model)
const Product = require('../models/productModel');
const productController = {
  getAllProducts: (req, res) => {
    Product.getAll((err, results) => {    // เรียกใช้ getAll จาก Model
      if (err) return res.status(500).json({ error: err.message });
      res.json(results);
    });
  },
  getProductById: (req, res) => {
    Product.getById(req.params.id, (err, results) => {
      if (err) return res.status(500).json({ error: err.message });
      res.json(results[0] || {});
    });
  },
  searchProducts: (req, res) => {
    const { keyword } = req.params;       // ดึง keyword จาก req.params
    Product.searchByKeyword(keyword, (err, results) => { ... });
  },
  createProduct: (req, res) => {
    const { name, price } = req.body;
    if (!name || price == null)
      return res.status(400).json({ error: 'name and price are required' });
    Product.create(req.body, (err, result) => {
      if (err) return res.status(500).json({ error: err.message });
      res.status(201).json({ message: 'Product created' });
    });
  },
  updateProduct:      // Product.update(id, req.body, ...) -> 'Product updated'
  softDeleteProduct:  // Product.softDelete(id, ...)     -> 'Product soft-deleted'
  restoreProduct:     // Product.restore(id, ...)        -> 'Product restored'
};
module.exports = productController;
```

- **Soft delete** = ไม่ลบแถวจริง แค่ `SET is_deleted = 1` แล้ว query ทุกตัวกรอง `WHERE is_deleted = 0`; **restore** = ตั้งกลับเป็น 0
- `?` + params array = parameterized query ป้องกัน SQL injection
- Search ใช้ `LIKE ?` กับ `'%keyword%'`
- Status: error DB → **500**; ข้อมูลไม่ครบ → **400**; สร้างสำเร็จ → **201**

View ใน Express.js

- **View** = ส่วนการแสดงผล (**Presentation Layer**) ของเว็บแอป ใช้แสดง HTML แบบไดนามิกที่เปลี่ยนตามข้อมูลที่ controller ส่งมา เป็นส่วนสำคัญของ MVC
- **Template Engine** = เครื่องมือแทรกข้อมูลลงใน HTML แบบไดนามิก (เช่น **EJS**, **Pug**)
- ติดตั้ง: `npm i ejs` (EJS = Embedded JavaScript templates); ตั้งค่า `app.set('view engine', 'ejs')` และผูก route หน้าเว็บ `app.use('/', webRoutes)` ใน index.js

Embedded JavaScript Templates (EJS)

Syntax	ความหมาย
`<p><%= ชื่อตัวแปร %></p>`	**แสดงตัวแปร** แบบ **escape** (ปลอดภัยจาก **XSS**)
`<%- htmlContent %>`	แสดงผล**โดยไม่ escape** = HTML ดิบ เช่น `<strong>เน้นข้อความ</strong>`
`<% if (loggedIn) { %> ... <% } %>`	ใส่ **JavaScript โดยไม่แสดงผลลัพธ์**
`<%- include('partials/header') %>`	**include ไฟล์** — partials/header.ejs ถูกแทรกเข้ามา

```
<!-- วนลูป forEach -->
<ul>
  <% รายการ.forEach(function(item) { %>
    <li><%= item %></li>
  <% }) %>
</ul>

<!-- เงื่อนไข -->
<% if (age >= 18) { %>
  <p>คุณบรรลุนิติภาวะแล้ว</p>
<% } else { %>
  <p>ยังไม่ถึงเกณฑ์</p>
<% } %>
```

```
// ส่งข้อมูลจากฝั่ง Server (index.js)
res.render('index', {
  title: 'หน้าแรก',
  user: { name: 'สมชาย' },
  products: ['สินค้า A', 'สินค้า B']
});
<!-- ใน EJS (index.ejs) -->
<h1><%= title %></h1>
<p>ชื่อผู้ใช้: <%= user.name %></p>
```

Lab ต่อ: แสดงสินค้าด้วย EJS

```
// views/products.ejs (ย่อ)
<h1>Product List</h1>
<div class="grid">
 <% products.forEach(product => { %>
  <div class="card">
   <img src="<%= product.image_url %>" alt="<%=
product.name %>" />
   <h3><%= product.name %></h3>
   <p><%= product.price.toLocaleString() %>
บาท</p>
  </div>
 <% }) %>
</div>

// routes/webRoutes.js
const express = require('express');
const router = express.Router();
const db = require('../configs/db');
const productController =
require('../controllers/productController');
router.get('/',
productController.getProductsView);
module.exports = router;

// controllers/productController.js (เพิ่ม)
getProductsView: (req, res) => {
  Product.getAll((err, results) => {   // เรียก
getAll จาก Product Model
    if (err) return res.status(500).json({
error: err.message });
    res.render('products', { products: results
}); // ส่งผลลัพธ์ไป view
  });
}
```

Static files (CSS)

- สร้าง folder `public/css/` แล้วย้าย CSS จาก `<style>` ไปไว้เป็นไฟล์
- เติม `app.use(express.static('public'))` ใน index.js เพื่อให้อ่านไฟล์ใน public ได้
- เปลี่ยนเป็นการ link stylesheet: `<link rel="stylesheet" href="/css/style.css">`

จำไว้: Route → Controller → Model → DB จากนั้น Controller → View (`res.render`) | Model ไม่ยุ่งกับ UI; View ไม่มี business logic; Controller ถือ logic ทั้งหมด

4 · Cookies, Sessions และ Password Hashing

JWT ไม่ออกสอบ

Cookie คืออะไร?

- **Cookie (HTTP Cookie)** = ข้อมูลขนาดเล็กที่ **Browser จัดเก็บ**ไว้เพื่อจดจำข้อมูลหรือสถานะการใช้งานเว็บไซต์ เช่น **Session ID** โดย Browser จะส่ง Cookie ที่ตรงเงื่อนไขกลับไปยัง Server พร้อมกับ HTTP Request ครั้งถัดไป
- Server ตั้งค่าด้วย header `Set-Cookie` (ใน response); Browser ส่งกลับใน header `Cookie`

ประโยชน์ของ Cookie

- **Authentication**: จดจำสถานะ login ไม่ต้อง login ใหม่ทุกครั้งที่เปลี่ยนหน้า
- **Session Management**: เก็บ Session ID ให้ Server ระบุผู้ใช้และจัดการ Session ได้
- **Personalization**: จดจำการตั้งค่า เช่น ภาษา ธีม การแสดงผล
- **Shopping Cart**: จดจำสินค้าในตะกร้า แม้จะเปลี่ยนหน้า
- **Analytics**: วิเคราะห์พฤติกรรม เช่น จำนวนการเข้าชม รูปแบบการใช้งาน
- **Advertising**: จดจำข้อมูลเพื่อแสดงโฆษณาที่ตรงความสนใจ

ข้อมูลที่ไม่ควรเก็บใน Cookies

- **Passwords**: ทั้ง Plaintext และ Password Hash
- **Credit Card Information**: เลขบัตร, CVV, ข้อมูลการชำระเงิน
- **Personal Sensitive Data**: เลขบัตรประชาชน, ข้อมูลสุขภาพ
- **Private Keys / API Secrets**
- **Database Credentials**: username/password ของ DB
- **User Roles / Permissions**: เช่น `isAdmin=true` (ผู้ใช้แก้เองได้!)
- **Confidential Business Data**

Demo: Session hijacking (the-internet.herokuapp.com)

1. เข้า `https://the-internet.herokuapp.com/login` → username `tomsmith` / password `SuperSecretPassword!`
2. เปิด DevTools → Application → Cookies → ลอง **Delete** cookie แล้วกด **F5** refresh → หลุดจากระบบ
3. Login ใหม่ แล้ว **Copy Value** ของ `rack.session`
4. เปิด **New Incognito Window** → เข้า `/secure` → ใส่ cookie `rack.session` ด้วยค่าที่ copy มา → **เข้าได้โดยไม่ต้อง login**

หมายเหตุ: rack.session value จะถูกส่งค่าใหม่มาแทนค่าเดิม; ถ้ากรอกค่า rack.session ไม่ถูกต้อง เว็บจะ Redirect ไป `/login`

Cookie attributes

Attribute	ผล
`secure: true`	จำกัดการส่ง Cookie ผ่าน **HTTPS** เท่านั้น (ยกเว้น localhost)
`httpOnly: true`	**ไม่ให้ JavaScript เข้าถึง** Cookie ผ่าน `document.cookie` (กันขโมยผ่าน XSS)
`sameSite: "none"`	**อนุญาต** Cookie ใน Cross-Site Requests (ต้องใช้คู่กับ Secure)
`"lax"` / `"strict"`	จำกัดการส่งข้าม site (กัน CSRF); ถ้าไม่ตั้ง Chrome/Edge ถือเป็น Lax (แต่ละ browser อาจต่างกัน)
Expires / Max-Age, Domain, Path	อายุ cookie (ไม่ตั้ง = session cookie หายเมื่อปิด browser) และ URL ที่จะได้รับ cookie

Project I — Session-cookie login

```
npm init -y
npm install express cors cookie-parser
```

- มี `index.html`, `client.js` (หน้าเว็บ) และ `server.js` — ปุ่ม: Login as Student / Lecturer / Admin และ Access Admin-Only Data

```
// client.js
adminBtn.addEventListener("click", () => {
  fetch("http://localhost:3000/adminData", {
    credentials: "include", // อนุญาตให้ส่ง Cookies แม้เป็น Cross-Origin
    headers: { "Content-Type": "application/json" },
  })
    .then((res) => res.text())
    .then((data) => (responsesDiv.textContent = data));
});
```

ไม่ระบุ credentials: "include"	ระบุ credentials: "include"

`GET /adminData HTTP/1.1`
`Host: localhost:3000`
	`GET /adminData HTTP/1.1`
	`Host: localhost:3000`
	`Cookie: sessionId=xxxxxxxxxx`

```
// server.js
const express = require("express");
const cors = require("cors");
const cookieParser = require("cookie-parser");
const crypto = require("crypto");
const app = express();
app.use(express.json());
app.use(cors({ origin: "http://127.0.0.1:5500",
credentials: true }));
app.use(cookieParser());

const USERS = new Map();
USERS.set("Admin",    { id: 1, username:
"Admin",    role: "Admin" });
USERS.set("Lecturer", { id: 2, username:
"Lecturer", role: "Lecturer" });
const SESSIONS = new Map();

app.post("/login", (req, res) => {
  const user = USERS.get(req.body.username);
  if (user == null) {
    res.sendStatus(401);       // เมื่อเป็น student
จะเข้าเงื่อนไขนี้
    return;
  }
  const sessionId = crypto.randomUUID();   //
UUID Version 4
  SESSIONS.set(sessionId, user);           // เก็บ
Session บน Server
  res.cookie("sessionId", sessionId, {
      secure: true,      // ส่งผ่าน HTTPS (ยกเว้น
localhost)
      httpOnly: true,    // ไม่ให้ JS เข้าถึงผ่าน
document.cookie
      sameSite: "none",  // อนุญาตใน Cross-Site
Requests
    })
    .send(`Authed as ${req.body.username}`);
});

app.get("/adminData", (req, res) => {
  const user =
SESSIONS.get(req.cookies.sessionId); // Backend
อ่าน Cookie
  if (user == null) {
    res.sendStatus(401);     // 401 Unauthorized
    return;
  }
  if (user.role !== "Admin") {
    res.sendStatus(403);     // 403 Forbidden
    return;
  }
  res.send("Authorized: You are viewing
restricted administrator controls.");
});
```

- รัน: `node server.js` แล้วคลิกขวา `index.html` → **Open with Live Server** (`http://127.0.0.1:5500/index.html`)
- Server `localhost:3000` กับ client `127.0.0.1:5500` คนละ origin → ต้องตั้ง **CORS** `credentials: true` ฝั่ง server และ `credentials: "include"` ฝั่ง client
- Login as Student → ได้ "Unauthorized" และไม่มี cookie; Login as Lecturer → "Authed as Lecturer" + มี `sessionId` ใน DevTools (Application → Cookies) แต่เรียก admin data ได้ **403**; Login as Admin → เข้าได้
- **Cookie เก็บแค่ session ID** ส่วนข้อมูล user/role อยู่บน **server** (`SESSIONS`) → ผู้ใช้ปลอม role ไม่ได้

แก้ปัญหา: Login เป็น Student แต่ยังเข้าถึงข้อมูล Admin ได้
(login เป็น Admin ก่อน แล้ว login Student ล้มเหลว → cookie Admin เดิมยังอยู่)

```
if (user == null) {
  SESSIONS.delete(req.cookies.sessionId); //
ลบ Session เดิมบน Server
  res.clearCookie("sessionId", {          //
ลบ Session Cookie ใน Browser
    secure: true, httpOnly: true, sameSite:
"none",
  });
  return res.sendStatus(401);             //
Login ไม่สำเร็จ
}
```

Project II — เก็บรหัสผ่านด้วย bcrypt

```
npm init -y
npm i express bcrypt
npm i --save-dev nodemon
// package.json เปลี่ยน "scripts" จาก "test": ...
เป็น
"scripts": {
  "start": "node server.js",
  "dev": "node --watch server.js"
},
// รัน
npm run dev
```

- **REST Client** (VS Code extension) = เครื่องมือส่ง HTTP Request ไป Server และดู HTTP Response **โดยไม่ต้องสร้างหน้าเว็บหรือเขียน Frontend ก่อน**

```
### Get all users
GET http://localhost:3000/users

### Create a new user
POST http://localhost:3000/users
Content-Type: application/json

{ "name": "Elon Musk", "password": "syntaxerror"
}

### Login user
POST http://localhost:3000/users/login
Content-Type: application/json

{ "name": "Elon Musk", "password": "syntaxerror"
}
```

ไฟล์ `request.rest`: แยก request ด้วย `###` แล้วกด **Send Request**; GET ครั้งแรกได้ `[]`, POST ได้ **201 Created**

```
// server.js เวอร์ชันแรก (เก็บ password ตรง ๆ - ไม่
ปลอดภัย)
const users = [];
app.get("/users", (req, res) =>
res.json(users));
app.post("/users", (req, res) => {
  const user = { name: req.body.name, password:
req.body.password };
  users.push(user);
  res.status(201).send();
}); // GET จะเห็น "password": "syntaxerror" แบบ
plaintext
```

Plaintext vs Encrypted vs Hash

วิธีเก็บ	ปัญหา
Plaintext (เช่น Student@123, Admin@2026)	ใครเปิด DB ได้ก็เห็นรหัสผ่านทั้งหมด
Encrypted	ถอดกลับได้ด้วย **Decryption key** → key หลุด = รหัสหลุดทั้งหมด
Hash (bcrypt hash)	ทางเดียว — **A hash cannot be reversed back into the original password** → ดีที่สุด

```
"P@ssW0rd"      =>
$2b$12$XqRwEneKhv2O4YwR5v.YQe8Sfn...
"wrongPassword" =>
$2b$12$Wftl0XnotlJ3EKmrMYc2guTb2t...
"P@ssW0rd" <= hash   // ย้อนกลับไม่ได้
```

bcrypt.hash(password, 10)

ส่วน	ความหมาย
Input password	รหัสผ่านต้นฉบับ เช่น MyPassword123
10 = Cost Factor	ยิ่งมากยิ่งช้า เพิ่มความยากในการเดารหัส — **ไม่ใช่ Salt และไม่ใช่ความยาวรหัสผ่าน**
bcrypt process	สร้าง **Salt แบบสุ่ม**, นำ Password + Salt มาคำนวณ, ทำงานช้าโดยตั้งใจ
Output	bcrypt hash ยาวรวม **60 ตัวอักษร** เช่น `$2b$10$...` สำหรับเก็บใน DB

```
// learn.js
const bcrypt = require("bcrypt");
const password = "MyPassword123";
// Hash สองครั้ง ใช้ Cost เท่ากัน แต่ salt ไม่เหมือนกัน เพราะสุ่มใหม่เสมอ
const hash1 = bcrypt.hash(password, 10);
const hash2 = bcrypt.hash(password, 10);
console.log(hash1 === hash2);
// false (ในสไลด์)
console.log(bcrypt.compare(password, hash1));
// true
console.log(bcrypt.compare(password, hash2));
// true
```

หมายเหตุ: `bcrypt.compare()` อ่าน **Salt และ Cost จาก Hash** ที่เก็บไว้โดยอัตโนมัติ จึงตรวจได้ถูกแม้ hash ต่างกัน
(ในสไลด์เขียนแบบไม่มี `await` — ถ้ารันตามนั้นจะพิมพ์ `Promise` ไม่ใช่ true; ของจริง `bcrypt.hash`/`compare` คืน Promise ต้อง `await`)

```
// server.js (เก็บแบบ hash)
const express = require("express");
const bcrypt = require("bcrypt");
const app = express();
app.use(express.json());
const users = [];

app.get("/users", (req, res) =>
res.json(users));

app.post("/users", async (req, res) => {
  try {
    const salt = await bcrypt.genSalt();
    const hashedPassword = await
bcrypt.hash(req.body.password, salt);
    console.log(salt);
    console.log(hashedPassword);
    const user = { name: req.body.name,
password: hashedPassword };
    users.push(user);
    res.status(201).send();
  } catch {
    res.status(500).send();
  }
});

app.post("/users/login", async (req, res) => {
  const user = users.find(user => user.name ===
req.body.name);
  // (สไลด์เขียน = แต่ที่ถูกคือ ===)
  if (user == null) {
    return res.status(400).send("Cannot find
user");
  }
  try {
    if (await bcrypt.compare(req.body.password,
user.password)) {
      res.send("Success");
    } else {
      res.send("Not Allowed");
    }
  } catch {
    res.status(500).send();
  }
});
app.listen(3000);
```

ผล: GET /users เห็น `"password": "$2b$10$.yFNYKz9..."` (hash) และ terminal แสดง salt กับ hash; POST /users/login ด้วยรหัสถูก → **Success**

จำไว้: cookie = เก็บแค่ ID | `httpOnly` กัน JS อ่าน, `secure` = HTTPS เท่านั้น, `sameSite` = กฎการส่งข้าม site | Hash ≠ Encrypt | Cost factor ≠ Salt | 401 = คุณคือใคร? 403 = ไม่มีสิทธิ์

5 · เครื่องมือที่ใช้ (Tools)

ภาพรวม: เครื่องมือ → ใช้ในหัวข้อไหน

Tool	คืออะไร	หัวข้อ
Node.js + npm	รัน JavaScript ฝั่ง server / จัดการ package	ทุกหัวข้อ
VS Code	Code editor (`code .` เปิด folder)	ทุกหัวข้อ
Jest	Unit testing framework	Jest
curl	ส่ง HTTP request จาก command line	API
Postman	GUI สำหรับส่ง/ทดสอบ API	API
Chrome DevTools	ดู Network, Cookies, Console ใน browser	API, Cookies
/docs (API documentation)	หน้าเอกสาร API ที่ลองเรียก endpoint ได้	API
Express	Web framework ของ Node.js	MVC, Cookies
EJS	Template engine (View)	MVC
mysql2, dotenv, uuid	เชื่อม MySQL / อ่าน .env / สร้าง ID	MVC
Docker Compose	รัน DB ของ lab จาก docker-compose.yaml	MVC
cors, cookie-parser, crypto	อนุญาต cross-origin / อ่าน cookie / สร้าง UUID	Cookies
bcrypt	Hash รหัสผ่าน	Cookies
REST Client (VS Code)	ส่ง request จากไฟล์ .rest	Cookies
Live Server (VS Code)	เสิร์ฟ HTML ที่ 127.0.0.1:5500	Cookies
nodemon / node --watch	รีสตาร์ท server อัตโนมัติเมื่อแก้โค้ด	Cookies

Node.js / npm

คำสั่ง	ความหมาย
`npm init -y`	สร้าง `package.json` ด้วยค่า default
`npm install express` / `npm i ...`	ติดตั้ง package (dependency) ลง `node_modules`
`npm install -D jest` / `--save-dev`	ติดตั้งเป็น **devDependency** (ใช้ตอนพัฒนา/ทดสอบเท่านั้น)
`npm test`, `npm run dev`, `npm start`	รัน script ใน `"scripts"` ของ package.json
`node server.js`	รันไฟล์ด้วย Node
`module.exports` / `require()`	ส่งออก / นำเข้า module (CommonJS)

ไฟล์ที่ได้: `package.json`, `package-lock.json`, `node_modules/`

Jest

- Unit testing framework ของ JavaScript (jestjs.io); ทางเลือกอื่น: Vitest, Node.js Native Test Runner, Mocha
- หาไฟล์ `*.test.js` อัตโนมัติ; รันด้วย `npm test` (script `"test": "jest"`)
- API หลัก: `test()`/`it()`, `describe()`, `expect().matcher()` (ดูหน้า 1)

เพิ่มเติม	ความหมาย
`jest --watch`	เฝ้าดูไฟล์ แล้วรันซ้ำเฉพาะ test ที่เกี่ยวกับไฟล์ที่แก้
`jest --watchAll`	รันซ้ำทุก test เมื่อมีอะไรเปลี่ยน
`jest --coverage`	เก็บและรายงาน test coverage
`jest --verbose`	แสดงผลแต่ละ test ตามโครงสร้าง suite
`beforeEach` / `afterEach`	รันก่อน/หลัง**ทุก** test (setup/teardown)
`beforeAll` / `afterAll`	รันครั้งเดียวก่อน/หลังทั้งหมด
`test.only(...)`	รันเฉพาะ test นี้ชั่วคราว

ใช้ผ่าน npm script: `npm test -- --coverage`

curl

```
curl https://randomuser.me/api/
# GET
curl -H "Accept: application/json" <url>
# ใส่ header
```

Flag	ความหมาย
`-H "Key: Value"`	ใส่ header (ใช้ในสไลด์)
`-X POST`	กำหนด method
`-d '{"a":1}'`	ส่ง body
`-i`	แสดง response headers + status
`-u user:pass`	Basic Auth

ถ้าไม่ส่ง Accept, icanhazdadjoke.com ตอบเป็น `text/plain`

Postman

- GUI สำหรับส่ง HTTP request และดู response; Lab ใช้ web version `web.postman.co` (ไม่ต้องติดตั้ง)
- ส่วนของ request: เลือก **Method + URL** → แท็บ **Params** (query string), **Authorization** (Basic Auth / Bearer Token), **Headers**, **Body** (raw → JSON) → กด **Send**
- Response แสดง status code, เวลา, headers, body
- ใน Lab: copy request จาก DevTools มาวาง แล้วแก้ body/description
- **Collections** = เก็บ request เป็นกลุ่ม; **Variables/Environments** เช่น `{{base_url}}`

Test scripts : เขียนใน **Scripts > Post-response** (เวอร์ชันเก่าเรียกแท็บ Tests) ใช้ `pm.test` คู่กับ `pm.response.to.have.status()` หรือ `pm.expect`

```
pm.test("Status code is 200", () => {
  pm.response.to.have.status(200);
});
pm.test("Successful POST request", () => {
  pm.expect(pm.response.code).to.be.oneOf([201,
202]);
});
pm.test("Response time < 300ms", () => {

pm.expect(pm.response.responseTime).to.be.below(300);
});
pm.test("has userId", () => {
  const data = pm.response.json();
  pm.expect(data).to.have.property("userId");
});
```

เทียบกับ Jest: `pm.test` ≈ `test()`, `pm.expect` ≈ `expect()` (Postman ใช้ไวยากรณ์ Chai)

Chrome DevTools (F12)

แท็บ	ใช้ทำอะไรในวิชานี้
Network	ดู request/response ที่หน้าเว็บส่ง (method, URL, status, headers, body) → ค้นหา endpoint ที่ไม่มีเอกสาร; คลิกขวา copy request ได้
Application → Cookies	ดู/แก้/ลบ cookie เช่น `sessionId`, `rack.session` และ flag HttpOnly/Secure/SameSite
Console	ดู log/error; ลอง `document.cookie` (cookie ที่เป็น HttpOnly จะไม่แสดง)

API documentation (/docs)

- เปิด `<Your-URL>/docs` → เห็นรายการ endpoint, ลองกดเรียกได้, มีปุ่ม **Authorize** ไว้วาง token หลัง login

Express ecosystem

Package	ใช้ทำอะไร / คำสั่งหลัก
express	`express()`, `app.get/post/put/delete`, `express.Router()`, `express.json()` (อ่าน JSON body), `express.static('public')`, `res.json / res.send / res.status / res.sendStatus / res.render / res.cookie / res.clearCookie`
mysql2	`mysql.createConnection({...})`, `db.query(sql, [params], callback)`
dotenv	`dotenv.config()` อ่านค่าจาก `.env` เข้า `process.env`
uuid	`const { v4: uuidv4 } = require('uuid')` → `uuidv4()`
ejs	`res.render('products', { products })`; ไฟล์ใน `views/`
cors	`cors({ origin, credentials: true })` อนุญาต request ข้าม origin พร้อม cookie
cookie-parser	`app.use(cookieParser())` → อ่าน `req.cookies.sessionId`
crypto (built-in)	`crypto.randomUUID()` สร้าง session ID (UUID v4)
bcrypt	`genSalt()`, `hash(pw, saltOrRounds)`, `compare(pw, hash)` (คืน Promise ต้อง await)

Docker Compose

- Lab MVC มี `docker-compose.yaml` สำหรับรัน MySQL; ค่าเชื่อมต่ออยู่ใน `.env` (DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME)
- คำสั่งทั่วไป: `docker compose up -d` / `docker compose down` (สไลด์ไม่ได้แสดงคำสั่ง)

VS Code extensions

Extension	วิธีใช้
REST Client (Huachao Mao)	สร้างไฟล์ `request.rest` เขียน request แยกด้วย `###` แล้วกด **Send Request** เหนือแต่ละ request → ผลแสดงในแท็บ Response (ไม่ต้องเขียน Frontend)
Live Server	คลิกขวา `index.html` → **Open with Live Server** → `http://127.0.0.1:5500` (reload อัตโนมัติ)

```
### Create a new user
POST http://localhost:3000/users
Content-Type: application/json

{ "name": "Elon Musk", "password": "syntaxerror"
}
```

ต้องมีบรรทัดว่างระหว่าง headers กับ body

nodemon / node --watch

- ทั้งสองรีสตาร์ท server อัตโนมัติเมื่อไฟล์เปลี่ยน; สไลด์ติดตั้ง `nodemon` (devDependency) แต่ script `"dev"` ใช้ `node --watch server.js` ที่มีใน Node เอง → ใช้อันใดอันหนึ่งก็ได้ (ถ้าใช้ nodemon: `"dev": "nodemon server.js"`)

เว็บที่ใช้ทดลอง

เว็บ	ใช้ทำอะไร
icanhazdadjoke.com, randomuser.me, pokeapi.co, httpbin.org, dog.ceo	API สาธารณะสำหรับฝึก GET ด้วย curl
api.mathjs.org	ตัวอย่าง REST API คำนวณ expression
countryinfo (oorsprong.org), Apollo Studio countries	ตัวอย่าง SOAP / GraphQL
github.com/djwester/todo-list-testing	แอป lab สำหรับ API testing (`make run-dev`)
base64decode.org	ถอด Base64 ของ Basic Auth
the-internet.herokuapp.com	Demo login / cookie (rack.session)
semver.org	Semantic Versioning

6 · นอกสไลด์ (Beyond the slides)

ทั้งหัวข้อนี้เป็นความรู้เสริม ไม่มีในสไลด์

1. req.params vs req.query vs req.body

ที่มา	ตัวอย่าง request	อ่านใน Express
req.params = ส่วนของ path (route parameter)	`GET /products/5` กับ route `/products/:id`	`req.params.id` → `"5"`
req.query = query string หลัง ?	`GET /tasks?status=In%20Progress`	`req.query.status` → `"In Progress"`
req.body = ข้อมูลใน body (POST/PUT)	`POST /users` + JSON `{"name":"Elon"}`	`req.body.name` (ต้องมี `app.use(express.json())`)

- ค่าจาก params/query เป็น **string เสมอ** (ต้องแปลงเองถ้าต้องการ number)
- ถ้าลืม `express.json()` → `req.body` เป็น `undefined`
- ใช้ทั่วไป: params = ระบุ**ตัว**ข้อมูล (id), query = **กรอง/ค้นหา**, body = **ข้อมูลที่จะสร้าง/แก้**

2. XSS และ CSRF

	XSS (Cross-Site Scripting)	CSRF (Cross-Site Request Forgery)
คืออะไร	ผู้โจมตีทำให้ **script ของเขารันบนเว็บเรา** (เช่น ใส่ `<script>` ในช่อง comment)	เว็บอื่น**หลอกให้ browser ของเหยื่อส่ง request** ไปเว็บที่เหยื่อ login อยู่ (browser แนบ cookie ให้อัตโนมัติ)
ผลเสีย	ขโมย cookie/session, แก้หน้าเว็บ, ทำ action แทนผู้ใช้	โอนเงิน/เปลี่ยนรหัส/ลบข้อมูล โดยผู้ใช้ไม่รู้
ป้องกัน (ในวิชานี้)	EJS `<%= %>` escape HTML; ใช้ `<%- %>` เฉพาะ HTML ที่ไว้ใจได้; cookie `httpOnly` → script อ่าน cookie ไม่ได้	cookie `sameSite: "lax"/"strict"` (lab ใช้ `"none"` เพราะ client กับ server คนละ origin); CSRF token

3. ทดสอบโค้ด async ใน Jest

bcrypt, database, fetch คืน **Promise** → test ต้องรอผล ไม่งั้น test จบก่อนและผ่านแบบผิด ๆ

```
// แบบ async/await (แนะนำ)
test("hash matches password", async () => {
  const hash = await
bcrypt.hash("MyPassword123", 10);
  const ok = await
bcrypt.compare("MyPassword123", hash);
  expect(ok).toBe(true);
});

// แบบ .resolves / .rejects (ต้อง await หรือ
return)
test("compare resolves true", async () => {
  const hash = await bcrypt.hash("pw", 10);
  await expect(bcrypt.compare("pw",
hash)).resolves.toBe(true);
});
test("rejects on bad input", async () => {
  await
expect(fetchUser(-1)).rejects.toThrow("Not
found");
});
```

- เทียบ: function ปกติที่ throw ใช้ `expect(() => fn()).toThrow()`; Promise ที่ reject ใช้ `await expect(promise).rejects.toThrow()`

4. PUT vs PATCH

	PUT	PATCH
ความหมาย	**แทนที่ทั้ง resource**	**แก้เฉพาะ field ที่ส่งมา**
field ที่ไม่ได้ส่ง	ถูกลบ/รีเซ็ต (ตามสเปก)	คงค่าเดิม
ตัวอย่าง	`PUT /users/1` ต้องส่งข้อมูลครบทุก field	`PATCH /users/1 {"email":"new@x.com"}` แก้แค่ email

เชื่อมกับ Lab API Testing: PUT แล้ว**ไม่ใส่ email ในbody → email ถูกลบ** — ตามหลัก PUT นี่คือพฤติกรรมของการแทนที่ทั้งก้อน แต่สไลด์นับเป็น **Bug** เพราะ API ควร**validate** แล้วปฏิเสธ (400) เมื่อ field จำเป็นหายไป หรือรองรับ PATCH สำหรับการแก้บางส่วน → ถ้าออกสอบ ตอบตามสไลด์ว่าเป็น Bug

ใน lab MVC: `updateProduct` ใช้ PUT และ `UPDATE ... SET name=?, price=?, ...` ทุก field → ถ้าส่งไม่ครบ field ที่ขาดจะกลายเป็น `NULL`